ASSIGNMENT -02

Q1 – Call by Value, by Reference and by Pointer 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Hostel Fee Concession Updater

The hostel office applies concessions (merit 10%, sports 15%, single parent 25%) to students' fees. A function written to apply the concession appears to work when its result is printed inside the function, but the fee stored for the student never changes.

Your Challenge

Write the concession function three ways – passing the fee by value, by reference and by pointer. Before running, predict the stored fee after each call, then verify.

Inquiry

Explain where each version stores its parameter, why only two of them change the caller's fee, and when a const reference parameter should be used instead.

Design Decision

Several concessions may apply to one student. Decide whether the function should change the fee in place or return the new fee, and defend your choice.

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>


void change(int a, int& b, int* c)
{
a += 10;
```

```
b += 10;

*c += 10;
}

int& bigger(int& p, int& q) { return p > q ? p : q; }

void show(int x, int y = 5, int z = 10) { std::cout << x + y + z << ' '; }

int main()
{
int a = 1, b = 2, c = 3;
change(a, b, &c);
std::cout << a << ' ' << b << ' ' << c << '\n';

bigger(a, c) = 100;
std::cout << a << ' ' << c << '\n';

show(1);
show(1, 2);
show(1, 2, 3);
std::cout << '\n';
}
```

My predicted output (write before running):

Explain which variables changed in each call and why, and which default arguments were

used by each call to show().

Complete the Code

Complete TODO 1 below so that the caller's fee is changed, and execute the program. #include <iostream>

```
void applyByValue(long long fee, int percent)
{
fee -= fee * percent / 100; // changes only the local copy
}

void applyConcession(long long& fee, int percent)
{
// TODO 1: Reduce the caller's fee by percent using integer arithmetic (call by
// reference)

}

int main()
{
long long hostelFee =
45000;
applyByValue(hostelFee,
10);
std::cout << "after call by value : " << hostelFee << '\n';
applyConcession(hostelFee, 10);
std::cout << "after call by reference: " << hostelFee << '\n';
}
```

Expected output:
after call by value : 45000

after call by reference: 40500

Explain why the call by value leaves the fee at 45000.

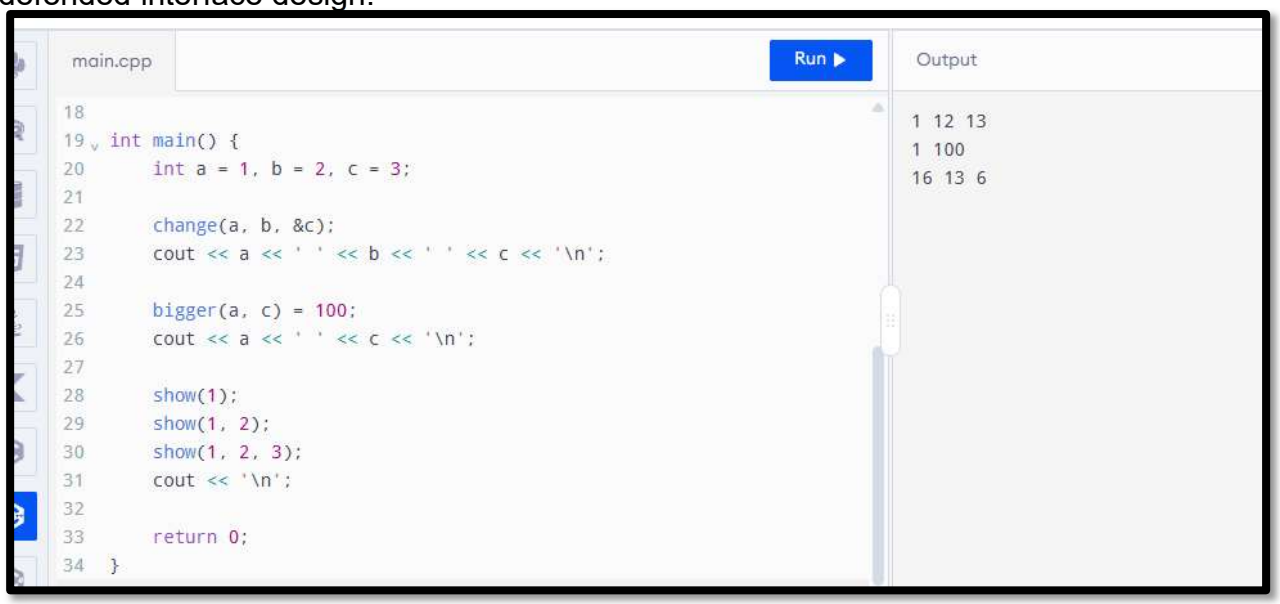
Alternative Concept and Complexity Analysis

If the concession function returned the new fee (long long applyConcession(long long fee, int percent)) instead of changing the fee through a reference, what would the solution be?

Analyse the time (O(1) in all versions) and the space copied per call for a long long and for a large Student object passed by value, by reference and by const reference.

Evidence Expected

Three implementations, predictions before execution, observed results and a defended interface design.



Q2 – Return by Reference and Its Dangers 4 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Examination Marks Register

The marks register keeps the marks of a class in an array. A function returns a reference to the highest mark so that a moderation of +2 can be applied directly to it. A second function returns a reference to a local variable.

Your Challenge

Implement both functions and show:

- How returning a reference lets the caller change the array element (and even use the call on the left of =)
- What the compiler warns about when a reference to a local variable is returned, and why using it is undefined behaviour
- The difference between returning a reference, returning a const reference and returning a copy

Changed Condition

The register is now stored in a std::vector that may grow. Explain why a reference obtained before the vector grows may become invalid.

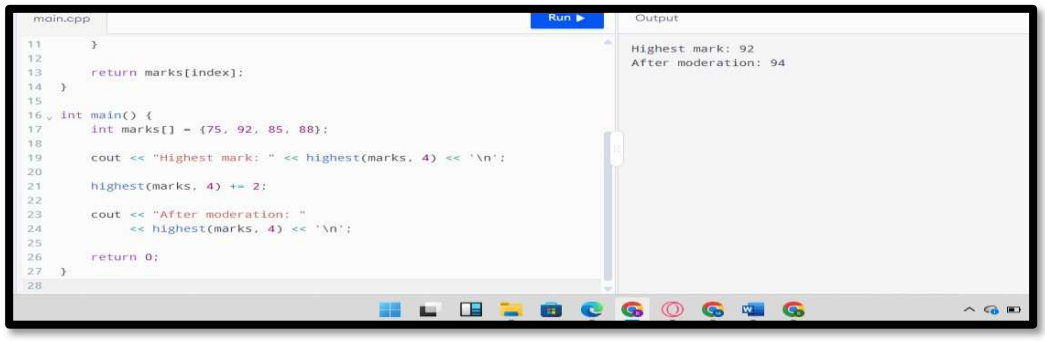
Alternative Concept and Complexity Analysis

If the highest mark were returned as an index (int) or as a pointer (int*) instead of a reference (int&), what would the calling code look like? Analyse the time to find the highest mark

(O(n) in all three) and the space (O(1)), and compare the safety of each when the array is later replaced by a growing vector.

Evidence Expected

Working functions, compiler-warning evidence, a correct explanation of dangling references and a justified response to the changed condition.



Q3 – Default Arguments and Function Overloading 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Courier Charge Calculator

A courier counter charges by weight; express delivery and insurance are optional. The developer wrote charge(double weight), charge(double weight, bool express) and charge(double weight, bool express = false, bool insured = false). The program no longer compiles.

Your Challenge

Explain the compiler error (ambiguous call). Then design the charge functions so that:

• Common calls are short (default arguments)

• Different kinds of input (weight in grams as int, weight in kg as double, a Parcel object) are handled by overloading

• No call is ambiguous

Before execution, predict which function each of six calls you design will select, including charge(2) and charge(2.0f).

Inquiry

Explain how the compiler chooses among overloaded functions (exact match, promotion, conversion), and why default arguments must be given from the right.

Alternative Concept and Complexity Analysis

If the courier charge took a single Parcel structure with optional fields (or a function template) instead of several overloaded functions with default arguments, what would the solution be? Analyse the time of overload resolution (at compile time, no run-time cost) and the space copied per call, and compare how each design handles a new option.

Evidence Expected

An explained ambiguity, a redesigned set of functions, predicted and observed

main.cpp

Run ▶

Output

```
29
30  double charge(Parcel p) {
31      return charge(p.weight, p.express, p.insured);
32  }
33
34  int main() {
35      cout << charge(200) << '\n';
36      cout << charge(2.0) << '\n';
37      cout << charge(2.0f) << '\n';
38      cout << charge(2.0, true) << '\n';
39      cout << charge(2.0, false, true) << '\n';
40
41      Parcel p = {3.0, true, true};
42      cout << charge(p) << '\n';
43
44      return 0;
45  }
```

```
10
100
100
130
120
200
```

Q4 – Specifying a Class with an Array Inside It 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Savings Account Mini-Statement

A bank wants each SavingsAccount object to remember its last 10 transactions for the mini-statement printed at an ATM. Older transactions are discarded.

Your Challenge

https://github.com/192472377/DSA0101

Specify the class with private data (account number, balance, an array of 10 transactions and a count) and public member functions deposit(), withdraw() and miniStatement(). Define some member functions inside the class and some outside using the scope resolution operator. Inquiry

Explain why the array must be private, what happens when the 11th transaction arrives if you shift the array versus use a circular index, and the cost of each.

Design Decision

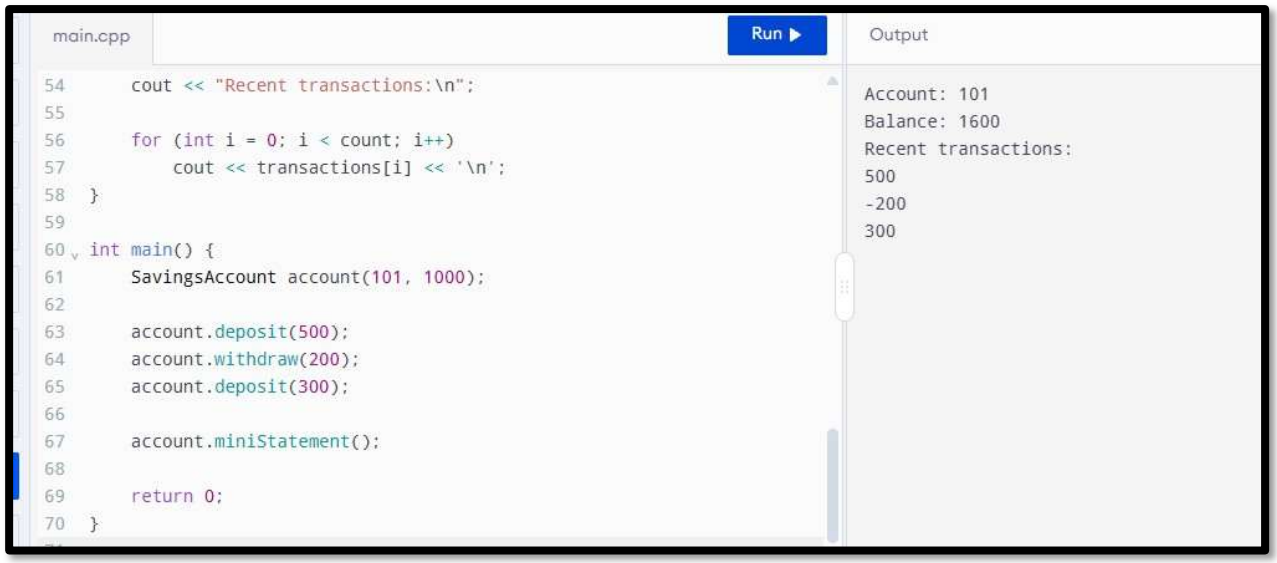
The bank now wants the last 50 transactions. Decide which design changes least and defend it.

Alternative Concept and Complexity Analysis

If the last 10 transactions were kept in a std::deque (push_back and pop_front), or in a circular buffer with a start index, instead of shifting a fixed array, what would the solution be? Analyse the time to add a transaction (O(k) for shifting k elements versus O(1)) and the space used in each design.

Evidence Expected

A complete class specification, member functions defined inside and outside the class, a justified handling of the 11th transaction and a defended design.



Q5 – Static Members and Arrays of Objects 7 Marks

Bloom's Level: K5 – Evaluate

Problem Context – College Bus Seat Tickets

The college bus has a fixed number of seats. Each BusTicket object must receive the next seat number, tickets beyond the capacity go to a waiting list, and the transport office must know how many seats have been issued without looking at any particular ticket.

Design Starting Point

You are given only the following principle:

Data that belongs to the whole class – not to any one object – should exist only once, however many objects are created.

Your Challenge

Convert this principle into a class design. Determine:

• Which members must be static, and where a static data member must be defined

• Why a static member function cannot use this or a non-static member

• How an array of BusTicket objects is created and initialised

• What happens to the count when tickets are created inside a loop or a function

Test Your Design

Create an array of tickets larger than the capacity and verify the seat numbers and the waiting list.

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>

class Token {
```

```
static int
next; int id;

public:
Token() { id = ++next; }
int get() const { return id;
}
static int issued() { return next - 100; }
};

int Token::next = 100;

int main()

{
Token
counter[3];
Token extra;
std::cout << counter[0].get() << ' ' << counter[2].get() << ' ' << extra.get()
<< ' ' << Token::issued() << '\n';
}
```

My predicted output (write before running):

Explain why the static counter starts at 100 and how many times the constructor ran. Complete the Code

Complete TODO 1 and TODO 2 below using the static members, and execute the program. #include <iostream>

```
#include <string>
class BusTicket {
static int issued; // shared by all BusTicket objects
static const int CAPACITY = 3;
int seat;
std::string
name;

public:
explicit BusTicket(const std::string& n) : seat(0), name(n)
{
// TODO 1: If fewer than CAPACITY tickets have been issued, increase issued and

// give this ticket the next seat number; otherwise leave seat as 0 (waiting
// list)

}
static int count()
{
// TODO 2: Return the number of seats issued so far

}

void print() const
{
std::cout << name << ": "
<< (seat ? "Seat " + std::to_string(seat) : std::string("Waiting"))
```

```
<< '\n';
}
};

int BusTicket::issued = 0; // definition of the static data member

int main()
{
BusTicket trip[4] = {BusTicket("Asha"), BusTicket("Bala"),
BusTicket("Chitra"), BusTicket("Dev")};
for (const BusTicket& t : trip) t.print();
std::cout << "Issued: " << BusTicket::count() << '\n';
}
```

Expected output:

Asha: Seat 1

Bala: Seat 2

Chitra: Seat 3 Dev: Waiting

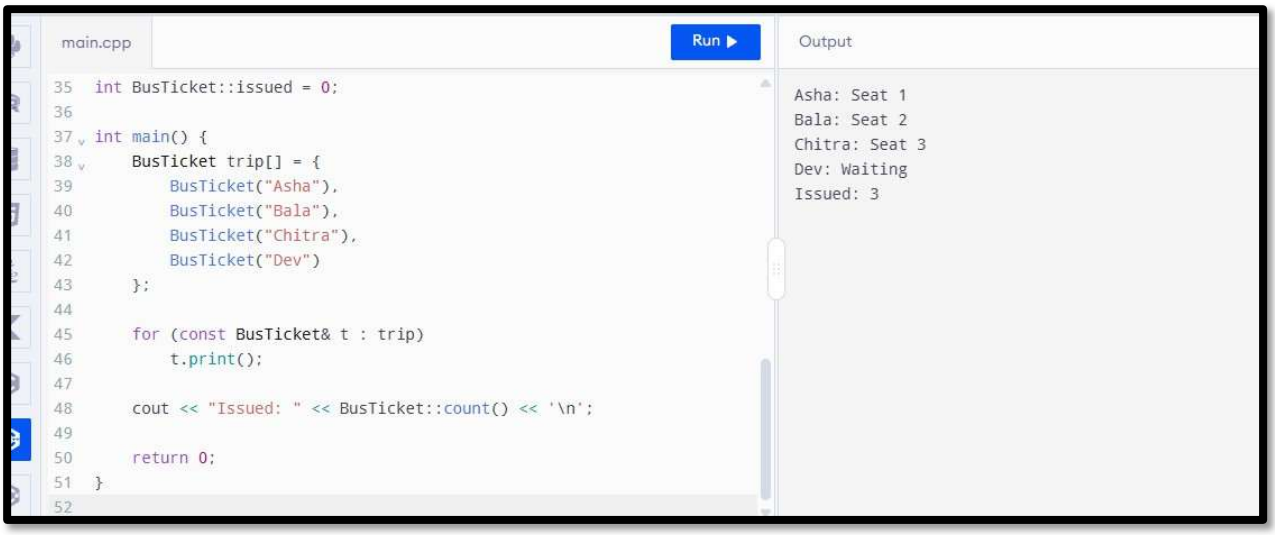
Issued: 3

Create the tickets inside a function that is called twice and explain how the count behaves. Alternative Concept and Complexity Analysis

If the seat counter were a global variable, or were kept in a separate TicketOffice object passed to every BusTicket, instead of a static data member, what would the solution be? Analyse the time (O(1) in all) and the space (one counter in all), and compare the encapsulation and the ability to run two buses at once.

Evidence Expected

A student-derived class design using static members, an array of objects, test evidence and a reasoned explanation of static members.



Q6 – Friend Functions and Inline Functions 8 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Train Journey Planner

A journey planner stores departure and arrival times in a Time class whose hours and minutes are private. It must compute the minutes between two times and decide which time is earlier, and these small functions are called millions of times while planning connections.

Your Challenge

Design the comparison. Determine:

• Why a friend function is needed when the function works on two objects equally

• How a friend function differs from a member function in how it is called and what it can access

• Whether the small functions should be inline, and what inline actually asks the compiler to do

• What would be lost if the private members were made public instead

Correctness Check

Test with times on either side of noon and the same time twice, and verify by hand. Growth Investigation

Measure the time for 10,000,000 calls with and without inline (compile with -O0 and with - O2), and explain the result.

Deployment Decision

Decide whether the friend functions should stay or be replaced by member functions and getters, and defend your decision.

Complete the Code

Complete TODO 1 and TODO 2 below inside the friend functions, and execute the program. #include <iostream>

```
class  Time  {
int h, m;

public:
Time(int hh, int mm) : h(hh), m(mm) {}
friend int minutesBetween(const Time& a, const Time& b);
friend bool earlier(const Time& a, const Time& b);
};

int minutesBetween(const Time& a, const Time& b)
{
// TODO 1: Using the private members h and m, return the absolute difference between
// the two times in minutes
```

```
}

bool earlier(const Time& a, const Time& b)
{

// TODO 2: Return true if a is earlier in the day than b

}

int main()
{
Time departure(9, 45), arrival(13, 10);
std::cout << minutesBetween(departure, arrival) << ' '
<< std::boolalpha << earlier(arrival, departure) << '\n';
}
```

Expected output:

205 false

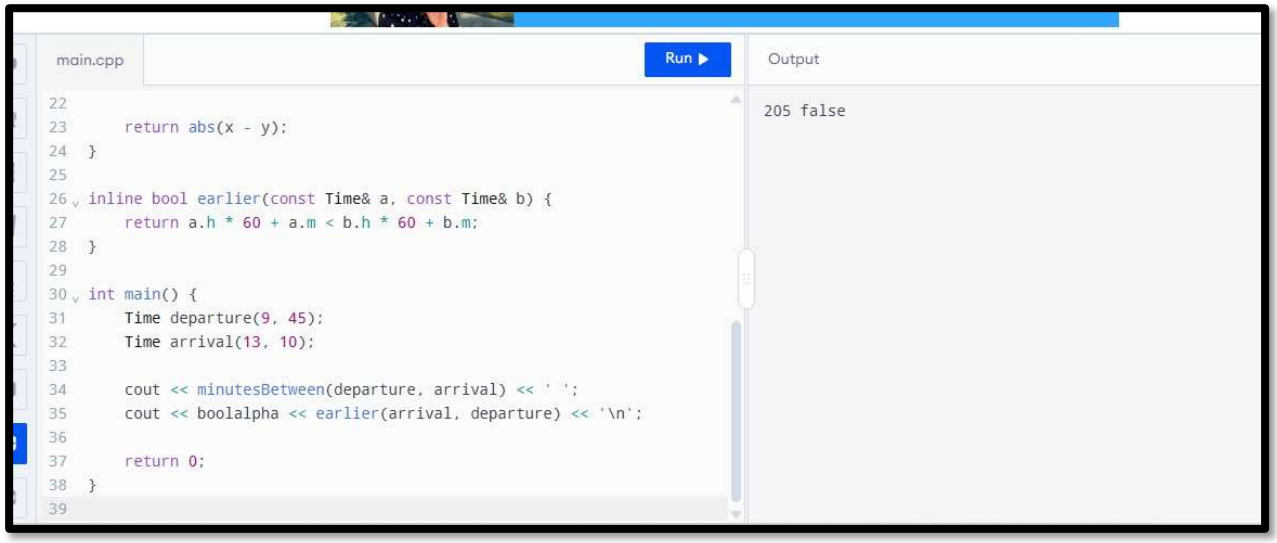
Test the same time twice and a pair of times on either side of noon. Alternative Concept and Complexity Analysis

If the comparison were written as member functions (a.minutesTo(b), a.isEarlier(b)) or with public getters instead of friend functions, what would the solution be? Analyse the time per call (the same after inlining, O(1)) and measure it at -O0 and -O2, and compare the encapsulation of the three designs.

Evidence Expected

A justified friend-function design, correctness tests, measured inline timings at two

optimisation levels and an evidence-based decision.



Q7 – Diagnose, Redesign and Validate 13 Marks

Bloom's Level: K5 – Evaluate

Review the evidence you generated while working on Q1–Q6. Select ONE program that shows a significant limitation as the data grows or the requirement changes.

You are not permitted simply to replace your code with a library facility and claim it is better. You must understand the problem first and then develop an appropriate redesign.

A. Diagnose the Limitation – 2 Marks

Using your own evidence, identify where the design fails or slows down and which characteristic of the original design causes it.

Evidence Expected

A precise diagnosis supported by evidence from your original program.

B. Define the Design Objective – 2 Marks State a measurable objective for the redesign. Evidence Expected

C. Design the Revised Solution – 4 Marks

Explain what you have changed in the class or function design, which C++ feature you use, whether extra memory is needed, and any new trade-off. Provide a class diagram or pseudocode.

Evidence Expected

A coherent redesign directly addressing the diagnosed limitation.

D. Predict Before Implementing – 1 Mark

Predict the improvement and the measure in which you expect it.

Evidence Expected

A clear, testable prediction.

E. Implement and Validate the Redesign – 2 Marks

Compare Original → Redesigned using the same tests and measurements. Evidence Expected

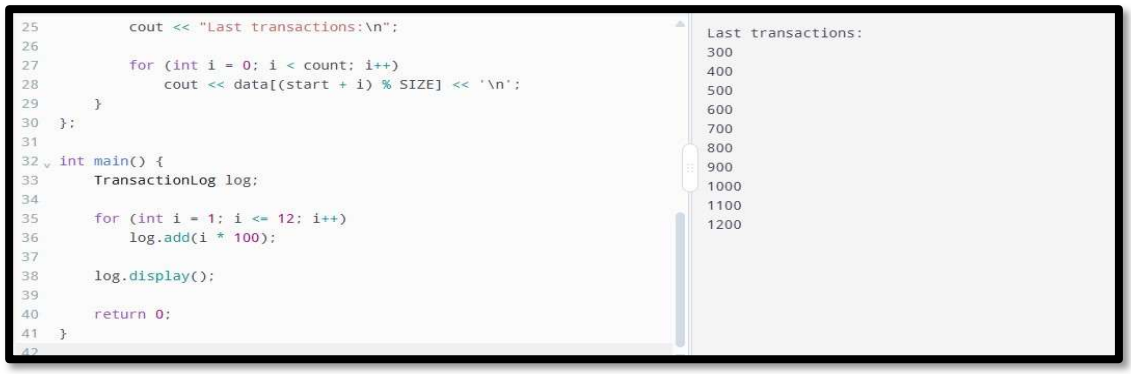
Comparable evidence for both versions.

F. Defend the Final Design – 2 Marks

Evaluate whether the objective was achieved, new costs, and where the original may still be preferable. A redesign that does not outperform the original is not automatically unsuccessful if you correctly explain why.

Evidence Expected

A critical, evidence-based evaluation and a defensible conclusion.



```
        cout << "Last transactions:\n";

        for (int i = 0; i < count; i++)
            cout << data[(start + i) % SIZE] << '\n';
    }
};

int main() {
    TransactionLog log;

    for (int i = 1; i <= 12; i++)
        log.add(i * 100);

    log.display();

    return 0;
}
```

```
Last transactions:
300
400
500
600
700
800
900
1000
1100
1200
```

https://github.com/192472377/DSA0101